

1. โปรแกรมข้างล่างนี้มีที่ผิดทั้งหมด 10 คำสั่ง จงหาว่ามีคำสั่งใดบ้างที่ผิด ห้ามตอบเกิน 10 คำสั่ง

```
#include<iostream>
using namespace std;
class C;
class A {
    int a;
    protected:
    A& f1(A x) { a=x; return *this; }
    public:
    A() { a=2; }
    A(int x) { a=x; }
    A f2() { return A(a+2); }
    A f3(A x) { return x.f2()+2; }
    A operator^(const A &x) {
        return A(a*x.a);
    }
    A* operator+=(const A x) {
        a+=x.a; return this;
    }
    int f4() { return a; }
    int f5(const A x) { return x.f2().f4(); }
    operator int() { return a; }
};

class B: public A {
    int b;
    protected:
    B g1(A x) { b+=x; return *this; }
    public:
    B(int a) { b=a; }
    A* g2(B &y) { return y+=*this; }
    A g3(const A x) { return f1(*this^x); }
    B g4(B &y) { return y.g3(*this).f4(); }
    B g5(A x) { return g1(*this+x); }
    B g6(const B y) { b+=y.f4(); return *this; }
    A* g7(B y) { b=++y.b; return this; }
    void g8(A &x, A k) { b=x.f1(k).f4(); }
    friend A h(C&, B&);
};
```

```
class C: public B {
    int c;
    public:
    C():B(3) { c=0; }
    C h1(int i) { c=g1(A(i)); return *this; }
    B h2(B &y) { return y.g1(c); }
    A* h3(A x) { return g4(*this)+x; }
    A* h4(A x) { return g7(*this); }
    B h5(B y) { return g5(*this^y); }
    int h6(C &z) {
        return (z.g3(*this)+=*this)->f4();
    }
    B h7(A x) { return B(f1(x).c); }
    C h8(B y) { return h1(y.f4()+c); }
};

A h(C &c, B &b) {
    A x(c.g5(b.b).b);
    C z(c.h1(b.g1(x)));
    x+=z.h3(x)->f4();
    return x.f1(z.f2());
}

int main() {
    A p;
    B q(p);
    C r;
    int k;
    k = r.h8(q).g5(p).f2();
    k = q.g3(p)^p;
    k = r.h4(p)->g3(p).f4();
    cout<<*(p+=p)<<endl;
    C s(r);
    k = s.h6(r)+p^q;
    k = s.h3(r.h5(q)).f4();
    cout<<*s.h4(*q.g7(r))<<endl;
    k = s.g4(q).g1(p).f4();
    cout<<p.f3(r.h8(q).f4())<<endl;
    cout<<q.g3(p).f1(s.f4())<<endl;
}
```

2. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 62 บรรทัด จงแสดงผลลัพธ์ทั้ง 62 บรรทัด

```
#include<iostream>
using namespace std;
class A {
    int a;
public:
    A() { a=1; cout<<"A "<<a<<endl; }
    A(int i) { a=i; cout<<"AA "<<a<<endl; }
    virtual ~A() { cout<<"a = "<<a<<endl; }
    virtual A set(int i) { a=i; return *this; }
    virtual void draw() {
        for (int i=0; i<a; i++) cout<<"#";
        cout<<endl;
    }
    operator int() { return a; }
};
class B: virtual public A {
    int b;
public:
    B():A(3) { b=2; cout<<"B "<<b<<endl; }
    B(int i):A(i+1) {
        b=i; cout<<"BB "<<b<<endl;
    }
    ~B() { cout<<"b = "<<b<<endl; }
    A set(int i) { b=i; return *this; }
    void draw() {
        for (int i=0; i<b; i++) cout<<"x";
        cout<<endl;
    }
    operator int() { return b; }
};
class C: virtual public A {
    int c;
public:
    C() { c=5; cout<<"C "<<c<<endl; }
    ~C() { cout<<"c = "<<c<<endl; }
    A set(int i) { c=i; return *this; }
    void draw() {
        for (int i=0; i<c; i++) cout<<"!";
        cout<<endl;
    }
    virtual operator int() { return c; }
};
```

```
class D: public B, public C {
    int d;
public:
    D():B(4) { d=2; cout<<"D "<<d<<endl; }
    D(int i) { d=i; cout<<"DD "<<d<<endl; }
    ~D() { cout<<"d = "<<d<<endl; }
    A set(int i) { d=i; return *this; }
    void draw() {
        B::draw();
        for (int i=0; i<d; i++) cout<<" / ";
        cout<<endl;
    }
    operator int() { return d; }
};

int main() {
    B b;
    C c;
    D d;
    A m[3] = { b, c, d };
    A *n[3] = { &b, &c, &d };
    for (int i=0; i<3; i++) {
        m[i].draw();
        n[i]->draw();
    }
    B *k;
    k = new D(4);
    cout<<*k<<endl;
    k->draw();
    C *h = new D(6);
    cout<<*h<<endl;
    h->draw();
    for (int i=0; i<3; i++) {
        cout<<n[i]->set((i+1)*2)<<endl;
        cout<<*n[i]<<endl;
        n[i]->draw();
    }
    delete h;
    delete k;
    cout<<"... CBA ..."<<endl;
}
```

3. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 26 บรรทัด จงแสดงผลลัพธ์ทั้ง 26 บรรทัด

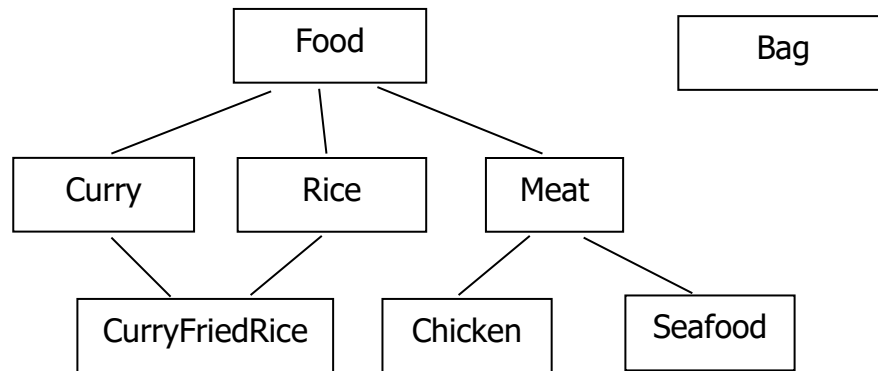
```
#include<iostream>
using namespace std;
class A {
    int a;
    public:
    A() { a=5; cout<<"A "<<a<<endl; }
    A(int i) { a=i; cout<<"AA "<<*this<<endl; }
    virtual ~A() { cout<<*this<<endl; }
    A(A &a) { this->a=a.a+1; cout<<"hey "<<a.a<<" "<<this->a<<endl; }
    virtual int get() { return a; }
    virtual A& operator()(int a) { this->a=a; return *this; }
    friend ostream& operator<<(ostream& o, A &x) {
        cout<<x.a<<" "<<x.get(); return o;
    }
    A& operator=(A &y) {
        cout<<"hi "<<get()<<" " + "<<y.get()<<endl;
        operator()(get() + y.get());
        return *this;
    }
};

class B: public A {
    int b;
    public:
    B():A(2) {
        b=3; cout<<"B "<<*this<<endl;
    }
    B(B &y):A(y) { b=y.b+2; }
    ~B() { cout<<*this<<endl; }
    int get() { return b; }
    A& operator()(int b) {
        this->b=b; return *this;
    }
};

class C {
    B b;
    public:
    C() { b(4); }
    C(C &c) { b = c.b; }
    int get() { return b.get(); }
    B operator()(int c) { b(c); return b; }
};

int main() {
    A n;
    A a(1);
    B b;
    C c;
    A* m[2] = { &a, &b };
    for (int i=0; i<2; i++)
        cout<<"SS "<<(*m[i])(i)<<endl;
    A a2=a;
    C c2(c);
    cout<<c(7).get()<<endl;
    cout<<"Bye Bye"<<endl;
}
```

จงสร้างคลาสตามรูปด้านล่าง โดยมีการรับทอดตามรูปและไม่เกิดความกำกวม โดยกำหนดให้การรับทอดทั้งหมดเป็นการรับทอดแบบ public ทุกคลาสให้เก็บแอททริบิวต์ทั้งหมดไว้ในส่วนของ private และกำหนดให้ Food และ Meat เป็น Abstract class โดยให้สร้าง string get() ใน Food และ Meat เป็น pure virtual function ส่วนใน subclass อื่นให้ไว้เก็บ name, type, veg ของคลาสตนเอง



แต่ละคลาสประกอบด้วยแอททริบิวต์และความหมาย ดังนี้ (ห้ามเปลี่ยนชื่อแอททริบิวต์)

	แอททริบิวต์	ความหมาย
Food	double price;	ราคา
Curry	int name; int spicyLevel; Meat **m; int num;	1.ต้มยำ 2.แกงเขียวหวาน 3.แกงเผ็ด ระดับความเผ็ด 1.น้อย 2.กลาง 3.มาก อาร์เรย์ขนาด num เก็บพอยเตอร์ที่ชี้ไปยัง Meat ขนาดของอาร์เรย์
Rice	int type;	1.ข้าวขาว 2.ข้าวกล้อง 3.ข้าวไรซ์เบอร์รี่
CurryFriedRice	int veg;	1.ใส่ต้นหอม 2.ใส่ผักชี 3.ใส่ต้นหอมและผักชี
Meat	int type;	1.เนื้อสด 2.ย่าง 3.นึ่ง 4.ทอด
Chicken	int name;	1.น่องไก่ 2.สะโพกไก่ 3.เนื้ออก
Seafood	int name;	1.กุ้ง 2.หอย 3.ปู 4.ปลา 5.ปลาหมึก
Bag	Food **food; int numFood; int type;	อาร์เรย์ขนาด num เก็บพอยเตอร์ที่ชี้ไปยัง Food ขนาดของอาร์เรย์ food 1.plastic bag 2.paper bag

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้ (ห้ามสร้างฟังก์ชันเพิ่มเติมถ้าโจทย์ไม่ได้สั่ง)
ทุกคลาส จงสร้าง

- default constructor ในการกำหนดค่าให้กับแอททริบิวต์ทุกตัว โดยให้กำหนดค่าเริ่มต้นเป็นอะไรก็ได้ที่สอดคล้องกับข้อมูลในตารางด้านบน และต้องกำหนดให้ m=0, num=0, food=0 numFood=0
- สร้างฟังก์ชัน set ในการกำหนดค่าให้กับแอททริบิวต์แต่ละตัว โดยที่
 - o กรณี m และ food ให้สร้างฟังก์ชัน setM(int, Meat*) และ setFood(int, Food*) ในการกำหนดค่าให้กับ m และ food โดยให้กำหนดค่าในอาร์เรย์ที่ละ 1 ค่า โดยระบุตำแหน่งที่เก็บในอาร์เรย์
 - o กรณี setNum และ setNumFood ให้คืนพื้นที่เดิมและจองพื้นที่ใหม่ให้กับ m และ food ด้วย

- ฟังก์ชัน get ในการรีเทิร์นแอทริบิวต์แต่ละค่า เช่น double getPrice(), int getType()
 - o สำหรับ getM(int i) และ getFood(int i) ให้รับค่า i เป็นตำแหน่งที่เก็บในอาร์เรย์ และให้รีเทิร์นค่าของสมาชิกในอาร์เรย์ตำแหน่งที่ i
 - o และให้สร้าง string get() โดยที่ใน Food และ Meat เป็น pure virtual function ส่วนใน class และ subclass อื่นให้รีเทิร์นค่า string ของ name, type, veg ของคลาสตนเอง
- ฟังก์ชัน show() โดยให้แสดงข้อมูลทุกอย่างของคลาสตนเองและของ superclass ออกทางจอภาพ
 - o กรณี Food และ subclass ให้ทำเป็น virtual function และใช้หลักการ dynamic binding
 - o กรณี Bag ให้ทำเป็นฟังก์ชันธรรมดา
- operator << ในการเรียกใช้ show() โดยให้ใช้หลักการ dynamic binding กรณี inherited

คลาส Food, Meat, Chicken, Seafood จงสร้าง

- constructor โดยรับข้อมูลจากพารามิเตอร์ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัวรวมทั้ง base class โดยให้ใช้ชื่อไปเรียก constructor ที่รับพารามิเตอร์ของ base class

คลาส Curry และ CurryFriedRice จงสร้าง

- copy constructor ในการก๊อปปี้ค่าทุกอย่างถูกต้อง

คลาส CurryFriedRice จงสร้าง

- ฟังก์ชันในการกำหนดค่าให้กับแอทริบิวต์ต่างๆ อย่างถูกต้องและมีความหมาย โดยใช้หลักการ function overloading ทั้งหมด 4 ฟังก์ชัน
- bool hasChicken() ให้รีเทิร์นว่าในข้าวผัดมี ไข่ เป็นส่วนประกอบอยู่หรือไม่

คลาส Meat จงสร้าง

- ฟังก์ชันในการนับจำนวนอ็อบเจกต์ที่เป็น Meat ว่า ณ ปัจจุบันมีจำนวนอ็อบเจกต์คงอยู่ในโปรแกรมทั้งหมดเท่าไร ถึงแม้ว่าจะไม่มีอ็อบเจกต์อยู่เลยก็ต้องสามารถบอกได้ด้วย ให้สร้างแอทริบิวต์และฟังก์ชันที่เกี่ยวข้องเพิ่มเติมได้

คลาส Curry และ Bag จงสร้าง

- destructor ในการคืนพื้นที่ของแอทริบิวต์ที่มีการจอง

คลาส Bag จงสร้าง

- operator!() ในการรีเทิร์นค่าผลรวมของราคาอาหารทั้งหมดในถุง ให้สร้างเป็น friend function และ pass by reference
- operator> ในการเปรียบเทียบว่าถุงแรกมีราคาอาหารรวมแพงกว่าถุงที่สองหรือไม่ ให้สร้างเป็นสมาชิกของคลาส
- ฟังก์ชัน int getTomyam() ในการหาว่าในถุงมีตำยำอยู่กี่จาน

ใน main

- จงสร้างอาร์เรย์เก็บอ็อบเจกต์โดยให้กำหนดค่าดังนี้
 - o chicken เป็นอาร์เรย์ขนาด 3 เก็บอ็อบเจกต์ของ Chicken โดยกำหนดค่าเป็น น่อง ไก่หนึ่ง, สะโพกไก่ย่าง และเนื้ออกสด และกำหนดราคาตามลำดับดังนี้ 30, 40, 25
 - o seafood เป็นอาร์เรย์ขนาด 5 เก็บอ็อบเจกต์ของ Seafood โดยกำหนดค่าเป็น กุ้งสด หอยย่าง ปูหนึ่ง ปลาทอด และปลาหมึกย่าง และกำหนดราคาตามลำดับดังนี้ 100, 200, 300, 400, 500
- จงสร้างไดนามิกส์อาร์เรย์ที่เก็บอ็อบเจกต์โดยให้กำหนดค่าดังนี้
 - o curry เป็นพอยเตอร์ที่ชี้ไปยังคลาส Curry จากนั้นให้จองพื้นที่เป็นอาร์เรย์ขนาด 3 เก็บอ็อบเจกต์ของคลาส Curry โดยกำหนดค่าตามลำดับดังนี้
 - curry[0] เป็นต้มยำ เผ็ดมาก ราคา 250 บาท ใส่ seafood[0], seafood[4]
 - curry[1] เป็นแกงเผ็ด เผ็ดกลาง ราคา 180 บาท ใส่ chicken[1]
 - curry[2] เป็นแกงเขียวหวาน เผ็ดน้อย ราคา 150 บาท ใส่ chicken[2]
 - o rice เป็นพอยเตอร์ที่ชี้ไปยังคลาส Rice จากนั้นให้จองพื้นที่เป็นอาร์เรย์ขนาด 3 เก็บอ็อบเจกต์ของคลาส Rice โดยกำหนดค่าตามลำดับดังนี้ ข้าวขาวราคา 25 บาท ข้าวกล้องราคา 30 บาท ข้าวไรซ์เบอร์รี่ราคา 35 บาท
- จงสร้างไดนามิกส์อ็อบเจกต์โดยให้กำหนดค่าดังนี้
 - o curryFriedRice1 ให้จองพื้นที่เก็บอ็อบเจกต์ของคลาส CurryFriedRice โดยกำหนดค่าเป็นข้าวกล้องผัดต้มยำ เผ็ดกลาง ใส่ seafood[2] และ seafood[4] ใส่ต้นหอม ราคา 120 บาท
 - o curryFriedRice2 ให้จองพื้นที่เก็บอ็อบเจกต์ของคลาส CurryFriedRice โดยกำหนดค่าเป็นข้าวขาวผัดแกงเขียวหวาน เผ็ดมาก ใส่ chicken[0] ใส่ผักชี ราคา 110 บาท
- จงสร้าง friedRice เป็นอาร์เรย์ขนาด 3 เก็บพอยเตอร์ที่ชี้ไปยังคลาส CurryFriedRice ให้จองพื้นที่ให้กับสมาชิกทุกตัวใน friedRice โดยให้เรียกใช้ copy constructor ของคลาส CurryFriedRice โดยกำหนดค่าให้เท่ากับ curryFriedRice1 จากนั้นให้เปลี่ยนค่าดังนี้
 - o เปลี่ยนค่าของ friedRice[1] เป็น แกงเผ็ด ใส่ต้นหอมและผักชี
 - o เปลี่ยนค่าของ friedRice[2] เป็น ข้าวไรซ์เบอร์รี่ ราคา 125 บาท
- จงสร้าง bag เป็นไดนามิกส์อาร์เรย์ ซึ่งเป็นพอยเตอร์ที่ชี้ไปยังคลาส Bag จากนั้นให้จองพื้นที่เป็นอาร์เรย์ขนาด 2 เก็บอ็อบเจกต์ของ Bag โดยให้กำหนดค่าดังนี้
 - o bag[0] เป็นถุงพลาสติก ใส่อาหาร curryFriedRice1, friedRice[0], seafood[2], seafood[3], curry[0], curry[1], rice[0], rice[2]
 - o Bag[1] เป็นถุงกระดาษ ใส่อาหาร chicken[1], seafood[4], curry[2], rice[1]
- จงแสดงค่าทั้งหมดใน bag โดยเรียกใช้ฟังก์ชัน show และแสดงค่าผลรวมของราคาอาหารในถุงทั้งหมดโดยเรียกใช้ operator!
- จงแสดงค่าทั้งหมดใน bag โดยไม่เรียกใช้ฟังก์ชัน show และ operator<< ที่ นศ. ทำไว้สำหรับคลาส Bag แต่ให้เรียกใช้ operator<< ที่ นศ. ทำไว้ในคลาสอื่นๆ และให้เรียกใช้ getTomyam() ในการแสดงจำนวนต้มยำในแต่ละถุงด้วย
- จงหาว่าในอาร์เรย์ bag มีแกงเขียวหวานอยู่กี่จาน และให้แสดงราคาของแกงเขียวหวานแต่ละจานออกทางจอภาพด้วย
- จงให้ curryFriedRice2 เรียกใช้ฟังก์ชันโอเวอร์โหลดดังที่ น.ศ. สร้างไว้ทั้ง 4 ฟังก์ชัน และให้แสดงค่าของ curryFriedRice2 โดยใช้ operator << ที่ นศ. ได้ทำเอาไว้ ทุกครั้งหลังเรียกใช้ฟังก์ชันในแต่ละครั้ง
- จงคืนพื้นที่ในลำดับที่ถูกต้อง